2026 开源鸿蒙大学生创新大赛-赛道一

项目开发设计文档

作品：KernAdapt

面向混合负载的自适应内核资源管控 Agent

学校名称：中国科学院大学

团队名称：OSKernel

队长：孙健康

指导教师：郑晨

队员 1：_____

队员 2：_____

2026 开源鸿蒙大学生创新大赛组委会

目 录

1 简介 **1**

1.1 背景 1

1.2 目的 1

2 设计描述 **2**

2.1 总体设计 2

2.2 实现思路 2

2.3 系统结构 3

2.3.1 模块划分 3

2.3.2 系统架构说明 3

2.3.3 文件结构 3

2.4 模块功能描述 4

2.4.1 KernAdapt 控制面功能描述 4

2.4.2 LockCoop 功能描述 5

2.4.3 FlowCausal 功能描述 5

2.5 业务/实现流程说明 6

2.5.1 用例 1：数据库资源配置与锁交接处理 6

2.5.2 用例 2：工具调用返回与会话服务核算 7

2.6 接口描述 8

2.6.1 调用接口 8

2.6.2 提供接口 8

2.7 UI 设计 9

2.8 OpenHarmony 系统接入设计 9

2.8.1 系统位置与职责 9

2.8.2 内核、工具链与板级条件 9

2.8.3 同步语义与有效依赖 10

2.8.4 权限与异常处理 10

2.9 运行方式与评价方法 11

2.9.1 使用入口与输出 11

2.9.2 指标与实验条件 11

3 其他 **12**

3.1 成员分工 12

3.2 困难与思考 12

3.3 参考 12

1 简介

1.1 背景

数据库、存储服务、代码工具与人工智能推理共置运行时，应用性能既取决于获得的资源，也取决于关键依赖能否及时推进。持锁者得不到 CPU 会阻塞等待者；工具工作流在检索、编译、测试等阶段切换时，真正影响返回的进程也随之变化。CPU 利用率、运行队列长度等聚合指标不足以单独判断应当推进哪个任务、将其放在哪里，以及采用何种资源配置。

本作品参加开源鸿蒙系统与技术创新赛道，围绕应用进展关系与内核资源分配展开设计。锁交接、工具返回和多核拓扑共同影响服务表现，项目将这些关系转化为能够观测、配置和评价的系统机制。

1.2 目的

本文提出 KernAdapt 自适应内核资源管控系统。控制面根据运行状态与用户目标预测组合资源配置对吞吐、延迟及共存任务的影响，比较候选方案并评估选择风险；执行模块检查权限、作用范围和资源预算，通过实测结果决定保留调整或恢复原配置。模型提出资源方案，内核完成逐任务调度，二者通过结构化动作及执行记录交互。

本文按照官方开发设计文档模板说明总体设计、模块职责、实现流程、接口和交互界面。配套《KernAdapt 完整技术报告》独立呈现八章技术论证、算法、公式、实验图表与参考文献，二者共同构成项目文档。

已有实验从不同层次评价系统：KernAdapt 在 560 个实验组上的平均 regret 为 0.04357，相对因子化 GBDT 降低 30.95%；LockCoop 在 openEuler ARM64 节点 0 的 256 线程 MariaDB OLTP 实验中，相对同机 `scx_simple` 达到 5.30 倍吞吐；FlowCausal 在 4 CPU、8 工作流的三轮交错回放中将平均完成时间降低 7.32%，全部完成时间增加 2.65%。各项结果的环境、基线、指标与分析分别见完整技术报告第四至七章。

完整技术报告阅读索引

阅读内容	对应章节与正文页码
研究问题与贡献	完整技术报告第一章（正文第 1 页起）。
相关工作与技术基础	完整技术报告第二章（正文第 4 页起）。
负载特征与问题模型	完整技术报告第三章（正文第 8 页起）。
KernAdapt 方法与评价	完整技术报告第四章（正文第 17 页起）。
LockCoop 机制与实验	完整技术报告第五章（正文第 26 页起）。
FlowCausal 机制与实验	完整技术报告第六章（正文第 43 页起）。
跨场景分析与适用范围	完整技术报告第七章（正文第 61 页起）。
研究结论与展望	完整技术报告第八章（正文第 63 页起）。

表中页码采用完整技术报告的正文页码，电子文件页序见仓库文档导航；两份 PDF 使用各自的目录和页码。

2 设计描述

2.1 总体设计

针对两类具体依赖，本文设计了 LockCoop 与 FlowCausal 。LockCoop 维护等待者、具体锁、已确认持锁者及近期 CPU 的关联：LockBoost 在解锁前向持锁者提供有上限的时间片扩展，LockNest 在唤醒后根据同锁执行历史选择 CPU，分别缓解持锁者调度延迟和锁交接的局部性损失。FlowCausal 以控制组（cgroup）组织会话，通过工具调用标识和实际阻塞事件确认工具返回所依赖的进程，在会话份额内安排短时间片提前运行，并将实际 CPU 使用计入所属会话。

系统采用用户态控制面与内核数据面分工：控制面负责运行状态、用户目标、候选方案及执行管理；内核数据面负责逐任务调度、锁交接状态、返回关系与机制计数。两层通过结构化参数、BPF 状态和执行记录连接。模型比较资源配置，逐任务调度由确定性内核路径完成。

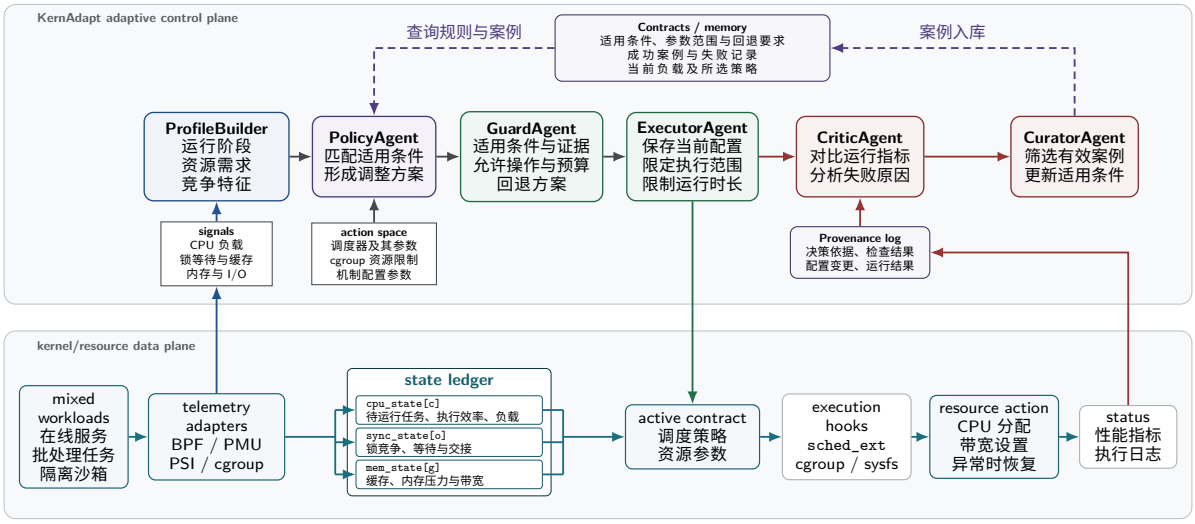


图 2.1: KernAdapt 控制架构。

控制面模型、学习与决策算法及评价见完整技术报告第四章（正文第 17 页起）。

2.2 实现思路

针对这些局限，本项目从用户可见进展而不是应用名称出发，研究进展关系、资源退化路径与内核动作之间的联系。本文将多种专用机制的在线选择表述为部分可观测状态下的受约束决策问题，并设计 KernAdapt 自适应内核资源管控系统，说明机制适用条件、调度动作、直接指标和应用性能之间的关系。在数据面，本文提出 LockCoop ，将等待者–锁–持锁者–后继者及其 CPU/缓存一致性拓扑关系暴露给调度器，以 LockBoost 推进被依赖的持锁者，并以 LockNest 保持同锁交接在缓存一致性拓扑中的局部性；同时提出 FlowCausal ，通过工具调用代际与实际阻塞等待确认工具返回关系，将有界提前运行与会话公平服务结合。在线专家只负责形成可检查的瓶颈假设与候选

策略，逐任务调度和资源执行仍由确定性数据面与安全约束完成。

1. 将应用等待关系与任务身份、同步对象及 CPU 拓扑联系起来，区分竞争线索和可靠的依赖证据。
2. 针对锁释放前后的不同问题分别使用有界持锁者推进和同锁后继放置；针对工具返回使用会话内有界提前运行。
3. 根据状态、目标和动作参数比较完整资源方案，保留默认调度为合法候选。
4. 在权限、作用域、预算和恢复要求下执行动作，用应用指标、机制计数和共存任务服务共同解释结果。

2.3 系统结构

2.3.1 模块划分

模块	软件层次	主要职责
KernAdapt 控制面	用户态 Python、SQLite 与 HTTP 服务	形成状态摘要、比较方案、检查动作、执行与恢复、记录运行。
LockCoop	用户态 loader / 同步观测库与内核 BPF	维护逐锁状态，执行 LockBoost 和 LockNest。
FlowCausal	用户态 C 登记程序与内核 BPF	登记会话和工具调用，确认返回关系，核算服务并安排提前运行。
结果与复现资料	文档、配置及归档记录	保存平台、工作量、基线、逐轮结果与机制分析。

表 2.1: 按功能相关性和执行层次划分模块。

2.3.2 系统架构说明

ProfileBuilder 汇总应用指标、锁等待、CPU 压力和环境信息；PolicyAgent 比较候选并形成 ActionPlan；Guard 检查参数、权限和作用范围；Executor 保存配置并实施受限动作；Critic 与经验管理流程解释结果和保存记录。LockCoop、FlowCausal 各自维护快速路径状态，不依赖用户态推理完成单次任务唤醒或 CPU 选择。

控制面源码中的 `AgentRun` 组织运行与检查点，`AgentScheduler` 维护角色任务，`PolicyEngine` 与 `CapabilityTable` 分别处理规则和角色能力，`DurableStore` 与 `ArtifactStore` 保存事件、状态与文件。`LockAgentHTTPServer` 和请求处理器把查询、任务操作及人工审核连接到运行时服务。上述实体位于控制面层；内核 BPF 状态和数据面回调分别位于两类机制模块。

2.3.3 文件结构

以下路径以现有源码仓库根目录为起点，细化到主要入口文件；本材料包中的源码索引提供完整工程入口。

文件或路径	作用
`src/agent/run_kernadapt_online.py`	KernAdapt 运行入口。
`src/agent/profile_builder.py`	构建应用状态摘要。

文件或路径	作用
src/agent/llm_policy_agent.py	形成与校验策略决定。
src/agent/runtime_guard.py	执行前后检查与约束判定。
src/agent/scx_executor.py	调度器执行与负载运行管理。
src/agent/agent_runtime.py	AgentRun、任务、能力、事件与持久化。
src/agent/runtime_service.py	HTTP 接口、任务队列与人工审核页面。
src/lockcoop/src/sched_ext/scx_lockcoop.c	用户态加载与参数设置。
src/lockcoop/src/sched_ext/scx_lockcoop.bpf.c	锁协同调度快速路径。
src/lockcoop/src/sched_ext/lockcoop_user_sync.c	用户同步观测。
src/flowcausal/src/user/flowd_v2.c	会话与工具登记。
src/flowcausal/src/user/flowevent_v2.c	工具事件入口。
src/flowcausal/src/user/scx_flowcausal_v2.c	调度器加载入口。
src/flowcausal/src/bpf/scx_flowcausal_v2.bpf.c	会话服务与返回关系调度。
src/flowcausal/src/include/flowcausal_v2.h	用户态与内核共享状态定义。
demo/README.md、docs/构建与运行.md	流程演示与运行说明。

2.4 模块功能描述

2.4.1 KernAdapt 控制面功能描述

标识与类型 KernAdapt；用户态资源管控子系统。

目的 在不同机制之间判断适用性、比较配置，并把运行调整组织成可检查、可追溯的过程。

功能列表 状态构建、候选选择、参数校验、角色能力检查、任务与租约管理、执行记录、效果评价和恢复。

处理 从应用状态和用户目标形成候选，检查动作是否满足机制及环境前提，保存原配置后实施受限动作；对照运行结果决定保留或恢复，并记录原因。

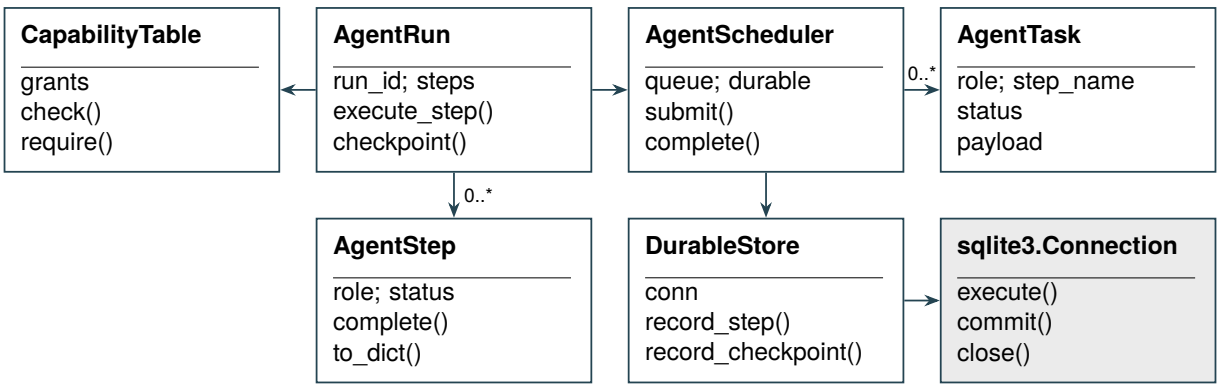


图 2.2: 运行时主要类及引用关系；省略次要关联。白底为项目类，灰底为 Python 标准库类，箭头表示对象持有的引用。

实体	功能分解
AgentRun、AgentStep	组织角色步骤、执行状态和检查点。
AgentScheduler、AgentTask	提交、分派和追踪角色任务及租约。
PolicyEngine、PolicyRule	按运行上下文评价策略规则。
CapabilityTable	检查角色请求的能力是否允许。
DurableStore、ArtifactStore	保存运行事件、状态记录与中间文件。
TypedEventBus、HumanGate	管理类型化事件及人工审核状态。

2.4.2 LockCoop 功能描述

标识与类型 LockCoop；用户态观测及内核调度子系统。

目的 同时处理锁释放前的持锁者调度延迟和释放后的共享数据局部性。

功能列表 规范化锁身份、维护等待者和已确认持锁者、有界时间片扩展、同锁 CPU 放置、锚点有效性检查、状态清理和机制计数。

处理 同步事件形成逐锁状态；LockBoost 按所有权、运行状态和预算决定推进；LockNest 按交接历史、拓扑、CPU 可用性和压力选择位置。记录失效或预算耗尽后退出相应优化路径。

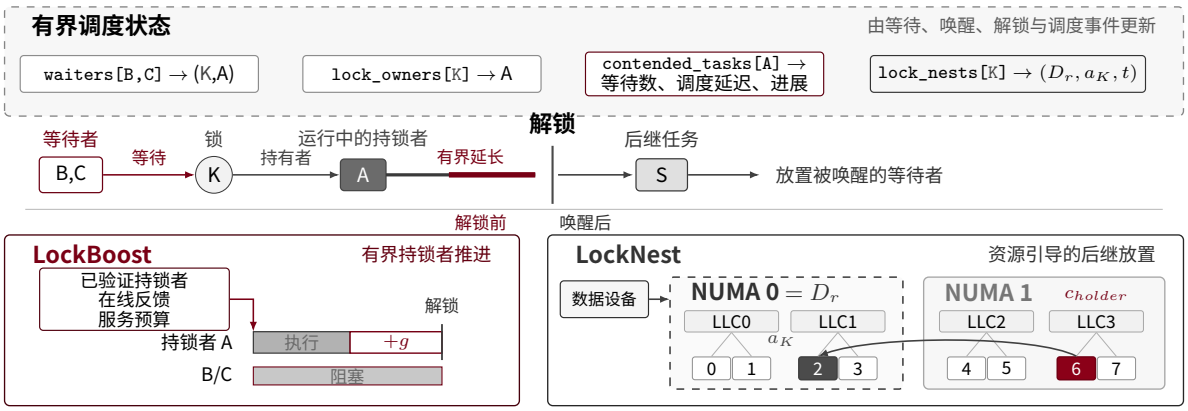


图 2.3: 逐锁状态与两类调度动作。

实现以 C/BPF 的 loader、观测库和调度回调划分职责。scx_lockcoop.c 配置与加载，scx_lockcoop.bpf.c 维护状态并执行调度，lockcoop_user_sync.c 提供对应同步观测。其状态机、预算和消融分析见完整技术报告第五章（正文第 26 页起）。

2.4.3 FlowCausal 功能描述

标识与类型 FlowCausal；会话登记与内核工作流调度子系统。

目的 根据实际阻塞确认工具返回依赖，在会话服务约束内改善等待对象的调度机会。

功能列表 会话/工具登记、调用代际跟踪、任务来源恢复、等待关系确认、会话 CPU 记账、有界提前运行及状态失效处理。

处理 用户态登记工具与会话，内核结合睡眠、唤醒和退出事件维护关系；会话外层分配服务，内层安排提前执行，实际 CPU 使用继续计入所属会话。

机制推导、会话公平性和回放实验见完整技术报告第六章（正文第 43 页起）。

共享头文件通过 flow_v2_session_config 表示会话配置，通过 flow_v2_tool_config 和 flow_v2_tool_runtime 表示工具配置与运行状态。任务来源保存在 flow_v2_task_meta，事件使用 flow_v2_event 表示。flowd_v2.c、flowevent_v2.c 与 BPF 程序通过共享状态协作。

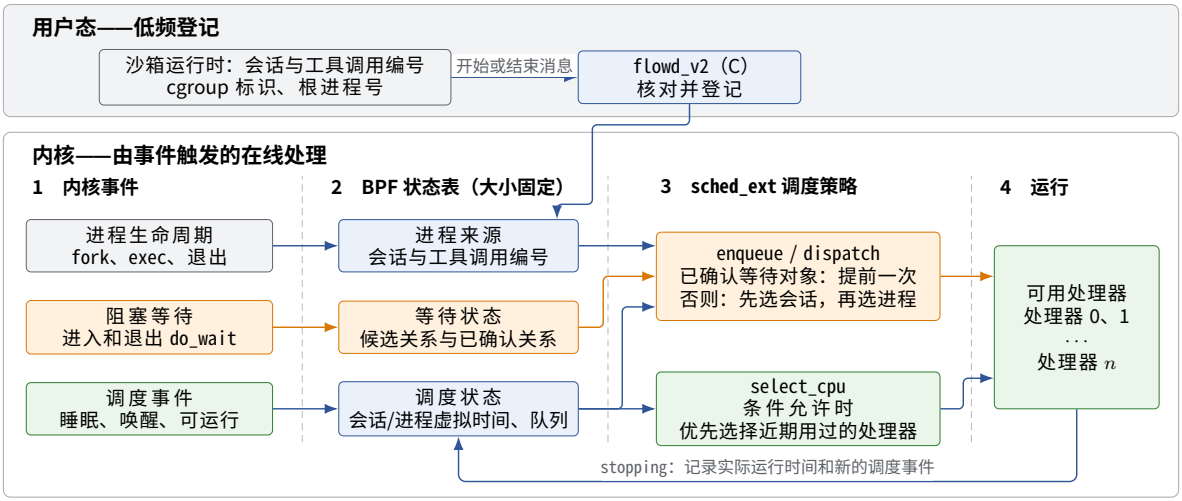


图 2.4: 工具登记、实际等待与会话调度结构。

2.5 业务/实现流程说明

2.5.1 用例 1：数据库资源配置与锁交接处理

顺序	参与实体	处理与交互
1	用户 / ProfileBuilder	指定服务目标与作用范围，形成应用指标、锁等待和环境摘要。
2	策略流程 / PolicyEngine	比较候选配置和默认策略，给出参数及依据。
3	Guard / CapabilityTable	校验参数、机制能力、角色权限、作用域和恢复条件。
4	Executor / loader	保存原配置，加载或配置调度机制，记录运行范围和期限。
5	同步观测 / BPF	数据库线程等待与交接时更新逐锁状态，执行有界推进或后继放置。
6	Critic / DurableStore	比较事务、后台服务和机制计数，记录结果并保留或恢复配置。

该流程涉及数据库工作线程、用户态观测/加载进程、内核调度事件和用户态评价流程。锁的互斥语义由同步原语保持，调度机制改变符合条件的任务执行时机和位置。

图2.5 展开流程中的执行与记录环节。图示为能力检查通过的路径；检查失败时不进入执行回调，执行异常则由 `execute_step()` 记录失败状态并重新抛出异常。权限检查、实际动作与记录职责分别落在对应对象上。

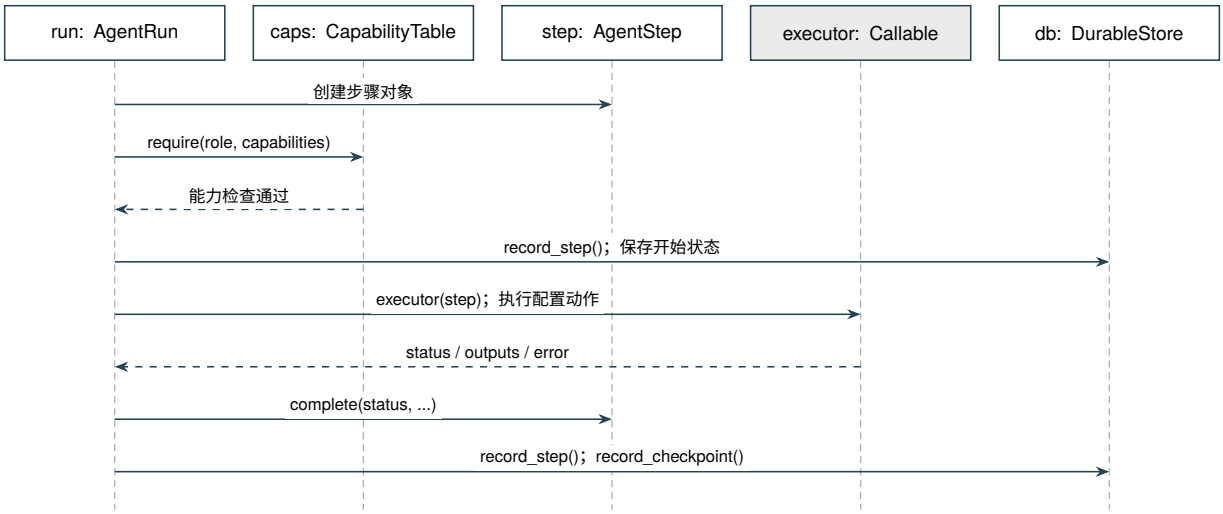


图 2.5: 配置动作经过运行时执行步骤的顺序。图中参与者是类实例或可调用对象；实线为调用，虚线为返回，灰底标出注入的外部执行回调。

2.5.2 用例 2：工具调用返回与会话服务核算

顺序	参与实体	处理与交互
1	工具运行器 / flowd	登记会话、cgroup 和本次工具调用身份。
2	工具进程 / flowevent	上报调用边界与来源，使进程活动能够关联到工具。
3	内核阻塞事件 / BPF	判断等待是否实际发生，并建立工具返回所依赖的对象。
4	会话调度状态	按份额、配额和并行宽度约束任务可获得的服务。
5	组内调度路径	为有效等待对象提供有限提前运行，累计实际 CPU 消耗。
6	唤醒、退出与调用结束	撤销失效关系，保存完成时间、服务及机制计数。

平均完成时间与全部工作流完成时间分别报告。CPU 可见的工具回放不将模型推理或远程 API 等待作为本地调度收益。

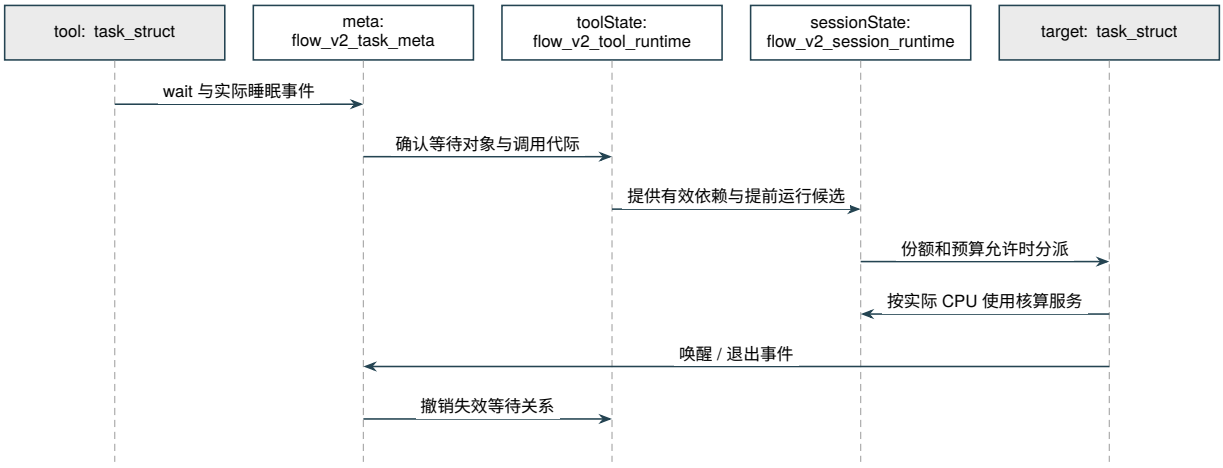


图 2.6: 工具返回处理的状态交互顺序。C/BPF 使用结构体对象组织状态；箭头表示事件处理产生的更新或调度动作，灰底为内核任务对象。

图2.6 对应已登记工具调用的运行过程。flow_v2_task_meta 记录任务来源与等待状态，flow_v2_tool_runtime 维护工具运行状态，BPF 源文件中的 flow_v2_session_runtime 核算会话服务。事件处理函数更新这些 C 结构体；该实现不使用面向对象语言的类继承。

2.6 接口描述

2.6.1 调用接口

接口	用途、输入与条件
Linux sched_ext/ BPF	加载调度器，处理任务入队、选择 CPU 与执行分派；依赖目标内核的 sched_ext、BTF 和所用接口。
libbpf / loader	设置数据面参数，创建或访问共享状态，连接 BPF 程序并获取计数。
同步观测接口	提供任务、锁及所有权事件；地址空间和 pthread ABI 按实际 libc 解释，普通唤醒者不直接作为持锁者。
cgroup / CPU 范围	确定会话归属、资源份额、配额及可用 CPU；修改与恢复保留相同作用范围。
应用结果采集	读取吞吐、尾延迟、完成时间及后台进度，保留来源、窗口与基线。

面向开源鸿蒙的接入同时涉及目标内核、板级组件和用户态语义。本文源码后端为 Linux sched_ext/eBPF；Linux 6.12 引入 sched_ext，具体实现按目标内核接口构建。glibc owner-word 路径与 OpenHarmony musl 普通 mutex 的锁字含义不同，持锁者推进依据经验证的所有权事件。

2.6.2 提供接口

运行时服务将查询、任务调度与审核功能提供给用户界面和外部执行者。以下接口来自 src/agent/runtime_service.py；内部字段与返回结构由对应服务函数维护。

接口	主要输入与输出
GET/health	返回服务状态、时间、存储位置和接口信息。
GET/tasks?checkpoint=...	输入运行检查点，返回任务、角色、状态和租约记录。
POST/tasks/submit	提交角色步骤与任务载荷，形成可追踪任务。
POST/worker/lease	领取可执行任务，记录执行者与租约。
POST/worker/heartbeat	延续任务执行状态和租约记录。
POST/worker/complete	上报任务结果并更新运行记录。
GET/memory/retrieve	按工作负载、调度器等条件检索经验。
GET/hitl	返回需要人工审核的运行与原因。
POST/hitl/approve	提交审核决定，并按请求参数控制恢复流程。
GET/ui/tasks	返回任务队列网页。
GET/ui/hitl	返回人工审核网页。

2.7 UI 设计

现有交互由命令行、运行时网页和结构化结果文件组成。命令行配置环境、目标与运行参数；网页集中展示任务及审核状态；JSON、事件记录与 SQLite 状态用于追踪和重放。

页面或入口	控件、布局和行为
任务入口页	以表格列出 Checkpoint 与 Action，点击 Open 进入指定运行的任务队列。
任务队列页	顶部显示 Checkpoint，表格列为 Task、Role、Step、Status、Lease Owner、Attempts。
人工审核页	表格列为 Status、Step、Reason、Checkpoint、Action；等待审核的记录提供操作按钮。
审核按钮	现有网页按钮为 Approve + dry-run resume，表单传递检查点与审核身份，使用 dry-run 恢复参数。
命令行与结果	运行入口提供参数与帮助；流程演示结果记录在 summary.json、guard 文件及 runtime 目录。

网页采用标题、检查点信息、带边框表格和行内操作按钮的布局。空队列显示相应提示；来自运行记录的文字先进行 HTML 转义。此处按源码中的界面和字段说明设计，界面展示的数值以实际运行记录为准。

2.8 OpenHarmony 系统接入设计

2.8.1 系统位置与职责

接入设计面向采用 Linux 内核的 OpenHarmony 系统。应用层提供业务目标和工具调用边界，用户态资源管理模块比较配置、记录执行与效果，内核数据面根据同步和调度事件处理具体任务。产品分支和板级组件提供运行环境，三类职责通过明确的接口连接。

系统层次	接入内容
应用与工具运行器	提供服务目标、吞吐与响应时间、会话标识和工具调用代际。
用户态控制面	构建状态、比较配置、检查执行范围、记录结果并管理恢复。
同步与任务观测	维护锁身份、可信持锁者、实际阻塞、唤醒、退出和状态生命周期。
Linux 调度后端	通过 sched_ext/BPF 执行持锁者推进、CPU 放置与会话服务核算。
产品与板级环境	提供驱动、CPU 拓扑、运行内核、构建工具和访问权限。

上述分层描述系统集成中的职责。仓库中的运行入口为 Python 控制面、HTTP 运行时服务、C 加载器与 BPF 程序。模型参与配置选择；锁交接与任务唤醒路径由内核机制直接处理。

2.8.2 内核、工具链与板级条件

OpenHarmony 内核接入同时涉及通用 Linux 内核、芯片平台代码和系统内核组件。官方 Linux 移植说明给出了这些部分的关系；具体组件与配置应按目标产品分支确定。板卡驱动、设备树、启动方式和系统组件需要与所选内核共同构成可运行的产品环境。

Linux 6.12 提供 sched_ext。本文归档实验包含 Linux 6.17 环境，程序使用的函数、结构体、调度回调和生成头文件与对应版本匹配。内核版本号只能帮助确定接口范围，不能单独证明程序与目标产品兼容。

环境信息	设计中的作用
产品与板卡	确定 OpenHarmony 分支、驱动、设备树、启动流程及系统组件。
内核与配置	确定 sched_ext、BPF、BTF、观测事件和调度回调。
编译目标与工具链	保持加载器、观测库、BPF 程序及目标 ABI 一致。
完整内核工具树	提供匹配的 `tools/sched_ext`、libbpf 和生成文件。
应用作用范围	记录 CPU、NUMA、cgroup、会话和可修改的配置。

项目构建脚本面向专用 Linux 工具工作树，负责加载器、观测库及调度程序的构建，不负责替换产品镜像或合并板级驱动。`docs/部署信息记录表.md` 用于保存具体部署的产品分支、板卡、内核、ABI 和作用范围。

2.8.3 同步语义与有效依赖

锁身份需要区分地址空间、同步对象与生命周期。相同虚拟地址可能来自不同进程；共享同步对象则需要按实际共享关系关联。对象复用、线程退出或记录过期后，相应状态必须失效。

OpenHarmony musl 普通 mutex 快速路径使用锁状态值，不能将普通锁字直接解释为 glibc 特定路径中的持锁线程标识。futex 唤醒只能提供唤醒关系，不能单独证明锁所有权。因此，调度动作按可获得的信息分别判定：具有可信所有权、实际等待和剩余预算时，LockBoost 才能延长相应持锁者的时间片；同锁执行历史与 CPU 条件有效时，LockNest 才能调整后继放置。缺少某类信息时，不据此猜测所有权或执行超出条件的动作。

工具工作流以会话和调用代际组织状态。进程亲缘关系用于恢复来源，实际阻塞用于确认返回依赖。调用结束、进程退出和记录超时撤销相应关系。工具等待对象获得的提前运行计入所属会话，配额、并行宽度和服务预算分别约束资源使用。

2.8.4 权限与异常处理

查询、策略选择和特权加载分别承担明确职责。配置执行前检查参数、角色能力、允许的任务范围和恢复条件；执行记录关联输入、决策、动作与结果。运行时服务的访问范围由部署环境约束，凭据保留在本地环境中。

条件	相应处理
所有权不明或记录过期	不触发依赖该证据的持锁者推进；清理或忽略失效状态。
目标 CPU 不可用或超出范围	放弃该放置建议，使用符合任务约束的普通调度路径。
时间片扩展预算耗尽	停止本轮扩展，避免依赖关系转化为无限服务。
工具退出或调用结束	撤销等待关系，保存会话服务及完成记录。
配置无法应用或约束不满足	记录失败原因，按对应动作恢复原配置。
调度器结束或触发内核退出	由 sched_ext 恢复默认调度，控制面另行处理自身修改的资源配置。

sched_ext 的退出行为与用户态恢复职责分开说明：内核机制恢复调度类别，控制面负责其管理的配置、运行状态与记录。具体异常路径以所用内核及模块实现为准。

2.9 运行方式与评价方法

2.9.1 使用入口与输出

仓库根目录提供离线控制流程，使用 Python 3.10+ 和标准库中的 SQLite。命令为 `python3demo/run_demo.py--outresults/demo`，以合成输入展示策略处理、角色能力检查和持久化记录；末尾读取归档实验结果，不重新测量数据库性能。

文件或入口	内容
`results/demo/summary.json`	处理步骤、策略决定、能力检查及归档结果摘要。
`results/demo/guard_*.json`	有效和无效输入的处理结果。
`results/demo/runtime/`	SQLite 状态、运行事件和文件索引。
`scripts/build-kernel.sh`	使用匹配内核工具树构建 BPF 调度器及用户态工具。
`docs/构建与运行.md`	环境准备、加载、退出、工作负载和数据入口。

实际加载使用兼容的运行内核、BPF/BTF 和管理权限。观测实际调度器名称与状态，结束时检查默认调度恢复。部署记录同时保存 CPU 范围、应用配置和内核版本。

2.9.2 指标与实验条件

评价围绕应用进展、共存服务和配置决策三个层次展开。数据库同时报告事务吞吐、p95 延迟和后台工作量；工作流同时报告平均完成时间与全部完成时间；控制面分别报告配置选择质量、动作效用、约束违反及恢复记录。

实验	数据来源与解释
LockCoop 多平台 OLTP	ARM64 汇总结合原始基线和后续五轮复测；x86 汇总结合固定配置基线和后续稳定资源域实验；RISC-V 为同批十轮对照。各平台分别保留操作系统、线程数与 CPU 放置。
KernAdapt 配置选择	560 组、报告共 2800 次动作执行，平均 regret 为 0.04357；regret 衡量所选配置与已测最佳候选的效用差距。
KernAdapt 独立决策	600 次决策，147 次调整，其中 114 次正收益；453 次保持 Linux。报告平均效用、区间及约束违反。
FlowCausal 工作流	4 CPU、8 工作流、每配置三轮交错回放；平均完成时间降低 7.32%，全部完成时间增加 2.65%。

ARM64 原始同批次在 256 线程下相对 `scx_simple` 的吞吐为约 4.82 倍；报告汇总的 5.30 倍使用后续 LockCoop 复测数据。二者所用批次不同，离线演示读取前者。详细数值、数据文件、基线和计算方式见 `docs/实验结果说明.md` 与 `docs/benchmarks.md`。

处理器架构与操作系统分别记录。ARM64、x86 和 RISC-V 的 Linux 实验用于说明对应环境中的结果；目标 OpenHarmony 产品的环境与结果按实际部署独立记录。工具回放的完成时间不包含完整交互会话中的在线模型推理和远程 API 等待。

3 其他

3.1 成员分工

参赛团队为中国科学院大学 OSKernel，队长为孙健康，指导教师为郑晨。团队工作包括问题分析、技术路线设计、三项机制研究、系统实现、平台实验和结果分析。下表按研究内容说明团队工作。

工作	主要贡献
LockCoop	逐锁状态、持锁者推进、同锁放置、资源域锚定及多平台实验。
FlowCausal	工具返回关系、会话核算、有界提前执行与回放实验。
KernAdapt	状态与目标建模、组合配置选择、执行检查、恢复和效果分析。
工程与材料	实现组织、复现入口、数据归档、设计文档与技术论文。

3.2 困难与思考

依赖证据与相关性 高 CPU 利用率或锁等待本身不能说明何种调度动作有效。项目通过逐锁身份、实际阻塞和直接机制指标连接动作与应用结果。

运行时机与执行位置 持锁者推进和同锁局部性处理不同阶段的问题，分别用预算与拓扑条件限制，并通过消融解释作用。

响应与公平性 工具等待对象可以提前获得有限服务，但实际 CPU 使用仍计入所属会话；平均响应、全部完成时间和后台进度共同报告。

模型与内核时间尺度 用户态模型比较方案，确定性数据面执行调度。权限、作用域、租约和恢复路径将错误选择的影响限制在明确范围。

跨平台接口 内核版本、CPU 拓扑和 libc 同步语义共同决定可用事件与执行能力，结果保留具体平台和对照条件。

3.3 参考

1. 《KernAdapt 完整技术报告》：八章正文、图表、算法、实验和参考文献，与本文一并提交。本文与完整报告同属鸿蒙参赛材料目录，完整报告文件为 `docs/完整技术报告.pdf`。
2. LockCoop 技术论文与补充附录、KernAdapt 技术论文、FlowCausal 技术论文，见提交仓库 `papers/` 目录。
3. 现有源码仓库 README、构建与运行、模块接口及成员贡献说明，见提交仓库的源码索引与依赖说明。
4. 2026 开源鸿蒙大学生创新大赛赛道一开发设计文档与答辩模板，赛事入口：`https://competitions.atomgit.com/competition/2074791206352715777/intro`。
5. Linux sched_ext 文档：`https://docs.kernel.org/6.12/scheduler/sched-ext.html`。
6. OpenHarmony 内核移植说明与 mutex 实现：`https://github.com/openharmony/docs/blob/master/zh-cn/device-dev/porting/porting-linux-kernel.md`；`https://github.com/openharmony/third_party_musl/blob/master/src/thread/pthread_mutex_lock.c`。